Simon Sutoris 7542170

Task 1: Evaluation of Untyped Lambda Calculus

1.

$$\begin{aligned} & (\lambda x.\lambda y.y)\ (\lambda a.a\ a) \\ \to\ & \lambda y.y \end{aligned}$$

2.

$$\begin{aligned} & ((\lambda x.\lambda y.x\ y)\ (\lambda z.\lambda w.w))\ (\lambda a.a\ a) \\ \to\ & (\lambda y.(\lambda z.\lambda w.w)\ y)\ (\lambda a.a\ a) \\ \to\ & (\lambda z.\lambda w.w)\ (\lambda a.a\ a) \\ \to\ & \lambda w.w \end{aligned}$$

Task 2: Church encoding

1.

$$\begin{aligned} \texttt{sum_leaves} &= \texttt{fold}\ (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ (\lambda v.v) \\ &= (\lambda f.\lambda g.\lambda t.t\ f\ g)\ (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ (\lambda v.v) \\ &\to (\lambda g.\lambda t.t\ (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ g)\ (\lambda v.v) \\ &\to \lambda t.t\ (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ (\lambda v.v) \end{aligned}$$

Applied to the example tree:

$$\begin{aligned} & \texttt{sum_leaves}\ (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2)) \\ \to^* & (\lambda t.t\ (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ (\lambda v.v))\ (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2)) \\ \to & (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2))\ (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ (\lambda v.v) \\ \to & (\lambda g.(\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ v\ ((\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2))\ (\lambda v.v) \\ \to & (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ v\ ((\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ v_1\ ((\lambda v.v)\ v_{11})\ ((\lambda v.v)\ v_{12}))\ ((\lambda v.v)\ v_2) \\ \to^* & (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ v\ ((\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ v_1\ v_{11}\ v_{12})\ v_2 \\ \to^* & (\lambda v.\lambda l.\lambda r.\texttt{plus}\ l\ r)\ v\ (\texttt{plus}\ v_{11}\ v_{12})\ v_2 \\ \to^* & \texttt{plus}\ (\texttt{plus}\ v_{11}\ v_{12})\ v_2 \end{aligned}$$

2.

$$\texttt{map} = \lambda f.\lambda t.t\ f$$

Applied to the example tree:

$$\begin{aligned} & \texttt{map}\ f'\ (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2)) \\ \to\ & (\lambda f.\lambda t.t\ f)\ f'\ (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2)) \\ \to\ & (\lambda t.t\ f')(\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2)) \\ \to\ & (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2))\ f' \\ \to\ & \lambda g.f'\ v\ (f'\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2) \end{aligned}$$

3.

$$\begin{aligned} \texttt{size} &= \texttt{fold}\ (\lambda v.\lambda l.\lambda r.\texttt{scc}\ \texttt{plus}\ l\ r)\ (\lambda v.c_1) \\ &\to (\lambda f.\lambda g.\lambda t.t\ f\ g)\ (\lambda v.\lambda l.\lambda r.\texttt{scc}\ \texttt{plus}\ l\ r)\ (\lambda v.c_1) \\ &\to (\lambda g.\lambda t.t\ (\lambda v.\lambda l.\lambda r.\texttt{scc}\ \texttt{plus}\ l\ r)\ g)\ (\lambda v.c_1) \\ &\to \lambda t.t\ (\lambda v.\lambda l.\lambda r.\texttt{scc}\ \texttt{plus}\ l\ r)\ (\lambda v.c_1) \end{aligned}$$

Applied to the example tree:

$$
\begin{array}{ll}
 & \texttt{size}\ (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2)) \\
\rightarrow^* & (\lambda t.t\ (\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ (\lambda v.c_1))\ (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2)) \\
\rightarrow & (\lambda f.\lambda g.f\ v\ (f\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2))\ (\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ (\lambda v.c_1) \\
\rightarrow & (\lambda g.(\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v\ ((\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v_1\ (g\ v_{11})\ (g\ v_{12}))\ (g\ v_2))\ (\lambda v.c_1) \\
\rightarrow & (\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v\ ((\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v_1\ ((\lambda v.c_1)\ v_{11})\ ((\lambda v.c_1)\ v_{12}))\ ((\lambda v.c_1)\ v_2) \\
\rightarrow^* & (\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v\ ((\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v_1\ c_1\ c_1)\ c_1 \\
\rightarrow^* & (\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v\ (\texttt{scc plus}\ c_1\ c_1)\ c_1 \\
\rightarrow^* & (\lambda v.\lambda l.\lambda r.\texttt{scc plus}\ l\ r)\ v\ c_3\ c_1 \\
\rightarrow^* & \texttt{scc plus}\ c_3\ c_1 \\
\rightarrow^* & c_5
\end{array}
$$

Task 3: Call-by-name lambda calculus

Inference rules

Call-by-value

$$(\lambda x.t_1)\ ((\lambda y.t_2)\ t_3) \rightarrow (\lambda x.t_1)\ [y \mapsto t_3]t_2$$

Call-by-name

$$(\lambda x.t_1)\ ((\lambda y.t_2)\ t_3) \rightarrow [x \mapsto ((\lambda y.t_2)\ t_3)]t_1$$

Evaluation

Call-by-value

$$
\begin{array}{ll}
 & (\lambda f.\lambda v.v\ v)\ ((\lambda x.x)\ (\lambda y.\lambda z.y))\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & (\lambda f.\lambda v.v\ v)\ (\lambda y.\lambda z.y)\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & (\lambda v.v\ v)\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & (\lambda v.v\ v)\ ((\lambda y.y)\ (\lambda y.y)) \\
\rightarrow & (\lambda v.v\ v)\ (\lambda y.y) \\
\rightarrow & (\lambda y.y)\ (\lambda y.y) \\
\rightarrow & \lambda y.y
\end{array}
$$

Call-by-name

$$
\begin{array}{ll}
 & (\lambda f.\lambda v.v\ v)\ ((\lambda x.x)\ (\lambda y.\lambda z.y))\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & (\lambda v.v\ v)\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & ((\lambda a.a\ a)\ (\lambda y.y))\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & ((\lambda y.y)\ (\lambda y.y))\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & (\lambda y.y)\ ((\lambda a.a\ a)\ (\lambda y.y)) \\
\rightarrow & (\lambda a.a\ a)\ (\lambda y.y) \\
\rightarrow & (\lambda y.y)\ (\lambda y.y) \\
\rightarrow & \lambda y.y
\end{array}
$$

Up- and downsides

Cbv is strict, while cbn is non-strict or lazy. Lazy evaluation can skip evaluation steps, when they are not needed. If an argument would loop, as for example in $\texttt{omega} = (\lambda x.x\ x)\ (\lambda x.x\ x)$, lazy evaluation could skip it, while strict evaluation would get stuck. Also, in a higher order language, a given function may produce side effects. In lazy-evaluation, due to skipping, in certain functions they may not be produced. In strict evaluation they are. Whether that is a good thing or not, is up for debate. Most higher order languages opt for cbv.

Task 4

No questions.